Imam Abdulrahman Bin Faisal University
College of Computer Science and Information Technology

ARTI 404 — Natural Language Processing

Lab 9 — Named Entity Recognition (NER)

Student Name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1

Name Entity Recognition (NER)

Setup

Run this cell first. It installs spaCy and downloads the small English model if it is not already installed.

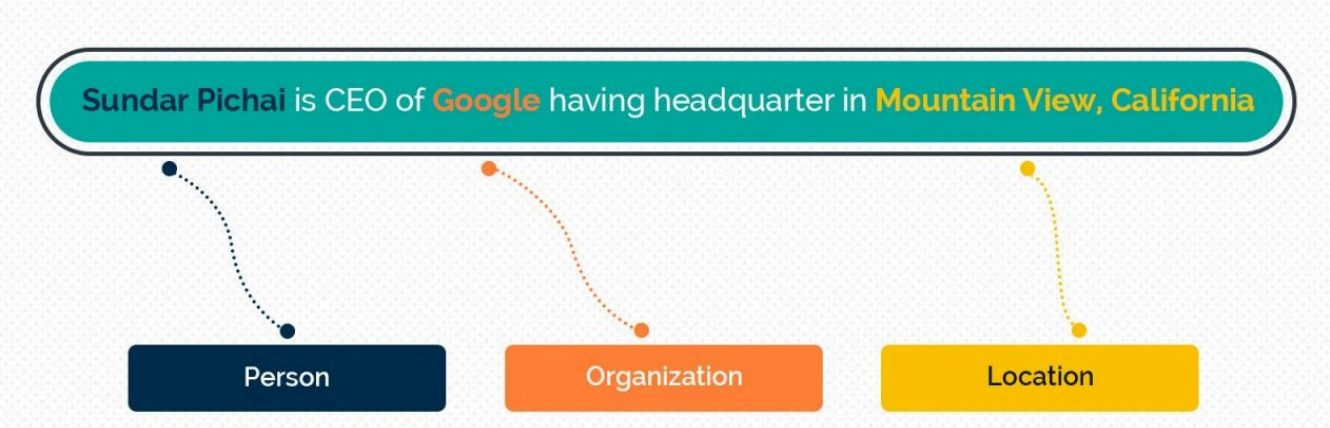
```
# Install the libraries used in this lab (safe to re-run)
import sys, subprocess
!{sys.executable} -m pip install -q spacy

# Download the small English spaCy model only if it is not installed yet
import spacy
try:
    spacy.load('en_core_web_sm')
    print("spaCy model 'en_core_web_sm' is already installed.")
except OSError:
    subprocess.run([sys.executable, "-m", "spacy", "download", "en_core_web_sm"])

print("Setup complete.")
```

```
spaCy model 'en_core_web_sm' is already installed.
Setup complete.
```

Name Entity Recognition (NER) is a fundamental task in NLP that focuses on identifying and classifying entities such as names of people, organizations, locations, dates, and more within a given text. NER plays a crucial role in various NLP applications, including information retrieval, question answering, and sentiment analysis.



At its core, it is just a two-step process:

1. Detecting the entities from the text
2. Classifying them into different categories

NER using spaCy

SpaCy offers robust support for Named Entity Recognition (NER). It can recognize a variety of named entities, such as persons, organizations, dates, locations, and more. Here's more information about using spaCy for NER:



TYPE	DESCRIPTION
PERSON	People, including fictional.
NORP	Nationalities or religious or political groups.
FAC	Buildings, airports, highways, bridge, etc.
ORG	Companies, agencies, institutions etc.
GPE	Countries, cities, states.
LOC	Non-GPE locations, mountain ranges, bodies of water.
PRODUCT	Objects, vehicles, foods, etc. (Not services)
EVENT	Named hurricanes, battles, wars, sports events, etc.
WORK_OF_ART	Titles of books, songs, etc.
LAW	Named documents made into laws.
LANGUAGE	Any named language.
DATE	Absolute or relative dates or periods.
TIME	Times smaller than a day.
PERCENT	Percentage, including "%".
MONEY	Monetary values, including unit.
QUANTITY	Measurements, as of weight or distance.
ORDINAL	"first" , "second" , etc.
CARDINAL	Nuerals that do not fall under another type.

When performing Named Entity Recognition (NER) using spaCy or other NLP tools, you'll encounter various entity types, and they are represented as labels. Here are some common entity types and their labels:

Entity Type	Label	Description	Examples
Person	`PERSON`	Represents names of people.	*John Smith*, *Barack Obama*
Organization	`ORG`	Names of companies, institutions, agencies, or other organized entities.	*Microsoft*, *United Nations*
Location	`GPE`, `LOC`	Geographical locations such as cities, countries, or landmarks.	*New York*, *Eiffel Tower*

Entity Type	Label	Description	Examples
Date	`DATE`	Calendar dates, time intervals, or relative dates.	*January 1, 2023*, *tomorrow*
Time	`TIME`	Clock times or time intervals.	*2:30 PM*, *5 minutes*
Money	`MONEY`	Monetary values, including currencies and amounts.	*$100*, *€50*
Percent	`PERCENT`	Percentage values.	*50%*, *10 percent*
Product	`PRODUCT`	Names of products or goods.	*iPhone*, *Coca-Cola*
Event	`EVENT`	Names of events or occurrences.	*World Cup*, *New Year's Eve*
Law	`LAW`	Legal references or statutes.	*Copyright Act*, *First Amendment*
Language	`LANGUAGE`	Names of languages.	*English*, *Spanish*
Nationality/Religion/Political Group	`NORP`	Nationalities, religious groups, or political affiliations.	*American*, *Buddhist*

In this example, we will use spaCy to perform NER on some sample text. Make sure you have spaCy installed; if not, you can install it using :

`pip install spacy`

```
# Import the spaCy library and load the English language model
import spacy
nlp = spacy.load("en_core_web_sm")

# Sample text for NER
text = "Apple Inc. was founded by Steve Jobs in Cupertino, California. It is known for
its iPhone and Mac products."
doc = nlp(text)

print(f"Entities recognized in text : {doc.ents}") #Return a tuple of entities
recognized in text

# Extract and print named entities
for entity in doc.ents:
    print(f"Entity: {entity.text}, Label: {entity.label_}")
```

```
Entities recognized in text : (Apple Inc., Steve Jobs, Cupertino, California, Mac)
Entity: Apple Inc., Label: ORG
Entity: Steve Jobs, Label: PERSON
Entity: Cupertino, Label: GPE
Entity: California, Label: GPE
Entity: Mac, Label: ORG
```

```python
# Iterate through recognized entities and print their details
for ent in doc.ents:
    print(f"Entity: {ent.text}")
    print(f"Label: {ent.label_}")
    print(f"Start Position: {ent.start}")
    print(f"End Position: {ent.end}")
    print(f"Entity Type: {spacy.explain(ent.label_)}")
    print("")
```

```
Entity: Apple Inc.
Label: ORG
Start Position: 0
End Position: 2
Entity Type: Companies, agencies, institutions, etc.

Entity: Steve Jobs
Label: PERSON
Start Position: 5
End Position: 7
Entity Type: People, including fictional

Entity: Cupertino
Label: GPE
Start Position: 8
End Position: 9
Entity Type: Countries, cities, states

Entity: California
Label: GPE
Start Position: 10
End Position: 11
Entity Type: Countries, cities, states

Entity: Mac
Label: ORG
Start Position: 19
End Position: 20
Entity Type: Companies, agencies, institutions, etc.
```

Create a custom Named Entity Model

In spaCy, you can add custom entities that have not been identified in the pre-trained models by training your own Named Entity Recognition (NER) model. This involves providing labeled examples of your custom entities and then fine-tuning the model.

Here is an example of how to train a custom NER model in spaCy:
https://spacy.io/usage/training#ner

```
from spacy.training.iob_utils import offsets_to_biluo_tags

# The text to annotate
text = "Apple Inc. was founded by Steve Jobs in Cupertino, California. It is known for
its iPhone and Mac products."

# --- Add custom entities annotations manually ---
annotations = [
    (text.find("Apple Inc."), text.find("Apple Inc.") + len("Apple Inc."), "ORG"),
    (text.find("Steve Jobs"), text.find("Steve Jobs") + len("Steve Jobs"), "PERSON"),
    (text.find("Cupertino, California"), text.find("Cupertino, California") +
len("Cupertino, California"), "GPE")]

# --- Step 2: Check entity alignment ---
doc = nlp.make_doc(text) #splitted to tokens
tags = offsets_to_biluo_tags(doc, annotations) #converts annotations into BILUO tags
(Begin, Inside, Last, Unit, Outside).
print("Alignment Tags:", tags[:10])
```

```
Alignment Tags: ['B-ORG', 'L-ORG', 'O', 'O', 'O', 'B-PERSON', 'L-PERSON', 'O', 'B-GPE', 'I-
GPE']
```

Training and saving the custom model

The annotations above only describe the entities; the model still has to be trained on them. The cell below trains the NER component on this single example and saves the result in a folder called `custom_ner_model`, which the next cell loads.

A separate copy of the pipeline is used for training so that the original `nlp` object keeps its pre-trained behaviour for the rest of the notebook.

```python
import spacy
from spacy.training import Example

# Load a separate copy of the model so the original `nlp` is not modified
train_nlp = spacy.load("en_core_web_sm")
ner = train_nlp.get_pipe("ner")

# Register the labels used in our annotations
for _, _, label in annotations:
    ner.add_label(label)

# Build one training example from the text and its annotations
example = Example.from_dict(train_nlp.make_doc(text), {"entities": annotations})

# Train only the NER component, leaving the rest of the pipeline unchanged
optimizer = train_nlp.resume_training()

with train_nlp.select_pipes(enable="ner"):
    for iteration in range(20):
        losses = {}
        train_nlp.update([example], drop=0.3, sgd=optimizer, losses=losses)
        print(f"Iteration {iteration + 1:>2}/20 - Loss: {losses['ner']:.4f}")

# Save the trained model to disk
train_nlp.to_disk("custom_ner_model")
print("\nThe model was saved in the folder: custom_ner_model")
```

```
Iteration  1/20 - Loss: 4.4265
Iteration  2/20 - Loss: 3.6960
Iteration  3/20 - Loss: 3.8192
Iteration  4/20 - Loss: 2.3746
Iteration  5/20 - Loss: 1.7231
Iteration  6/20 - Loss: 0.4382
Iteration  7/20 - Loss: 0.0397
Iteration  8/20 - Loss: 0.0095
Iteration  9/20 - Loss: 0.0151
Iteration 10/20 - Loss: 0.1226
Iteration 11/20 - Loss: 0.0005
Iteration 12/20 - Loss: 0.0039
Iteration 13/20 - Loss: 0.0000
Iteration 14/20 - Loss: 0.0001
Iteration 15/20 - Loss: 0.0000
Iteration 16/20 - Loss: 0.0000
Iteration 17/20 - Loss: 0.0014
Iteration 18/20 - Loss: 0.0000
Iteration 19/20 - Loss: 0.0000
Iteration 20/20 - Loss: 0.0000

The model was saved in the folder: custom_ner_model
```

Visualization

```python
# Load and use the custom model for entity recognition
custom_nlp = spacy.load("custom_ner_model")
doc2 = custom_nlp(text)

# Entity visualization
from spacy import displacy
displacy.render(doc2, style="ent", jupyter=True)
```

Apple Inc. **ORG** was founded by Steve Jobs **PERSON** in Cupertino, California **GPE** . It is known for its iPhone and Mac products.

Task 1

1. Ask a user to enter a sentence, and identify the named entities in the sentence.
2. Print the details of each entity in the sentence provided by the user.

```python
# Task 1: ask the user for a sentence and identify its named entities
import spacy

user_text = input("Enter a sentence: ")

doc_user = nlp(user_text)

print("\nSentence:", user_text)

if len(doc_user.ents) == 0:
    print("\nNo named entities were found in this sentence.")
else:
    print(f"\nNumber of entities found: {len(doc_user.ents)}\n")

    # Print the details of every entity
    for ent in doc_user.ents:
        print(f"Entity         : {ent.text}")
        print(f"Label          : {ent.label_}")
        print(f"Start Position : {ent.start}")
        print(f"End Position   : {ent.end}")
        print(f"Entity Type    : {spacy.explain(ent.label_)}")
        print("-" * 55)
```

```
Sentence: Ahmed studies at Imam Abdulrahman Bin Faisal University in Dammam since 2023.

Number of entities found: 3

Entity         : Imam Abdulrahman Bin Faisal University
Label          : ORG
Start Position : 3
End Position   : 8
Entity Type    : Companies, agencies, institutions, etc.
-------------------------------------------------------
Entity         : Dammam
Label          : GPE
Start Position : 9
End Position   : 10
Entity Type    : Countries, cities, states
-------------------------------------------------------
Entity         : 2023
Label          : DATE
Start Position : 11
End Position   : 12
Entity Type    : Absolute or relative dates or periods
-------------------------------------------------------
```

Task 2

Use the paragraph below to answer the following questions:

“This agreement is made and entered into by and between ‘Abc & Co.’ and ‘Bcd LLC’ for term 1 year starting from April 1, 2020, hereinafter collectively referred to as the Parties. Samuel P. Jackson in the place (New York) and on the date written below, with the following terms and conditions.”

1. Find any entity that has not been identified and add it.
2. Find the count of ‘PERSON’ and ‘ORG’ in the paragraph above.
3. Visualize the entities in the paragraph.

```python
# Task 2 - 1. Find any entity that has not been identified correctly and add it
paragraph = ("This agreement is made and entered into by and between 'Abc & Co.' and "
             "'Bcd LLC' for term 1 year starting from April 1, 2020, hereinafter "
             "collectively referred to as the Parties. Samuel P. Jackson in the place "
             "(New York) and on the date written below, with the following terms and "
             "conditions.")

doc3 = nlp(paragraph)

print("Entities recognized by the pre-trained model:\n")
for ent in doc3.ents:
    print(f"{ent.text:<22} {ent.label_:<12} {spacy.explain(ent.label_)}")

# The model made two mistakes:
#   - 'Abc & Co.' : the span wrongly included the word "between"
#   - 'Bcd LLC'   : labelled WORK_OF_ART instead of ORG
# Both company names are added manually as ORG entities
company_names = ["Abc & Co.", "Bcd LLC"]
new_entities = []

for name in company_names:
    start_char = paragraph.find(name)
    span = doc3.char_span(start_char, start_char + len(name), label="ORG",
                          alignment_mode="expand")
    new_entities.append(span)
    print(f"\nEntity added/corrected: '{span.text}' -> ORG")

# Keep the entities that do not overlap the corrected ones
kept = [e for e in doc3.ents
        if all(e.end <= s.start or e.start >= s.end for s in new_entities)]
doc3.set_ents(kept + new_entities, default="outside")

print("\nEntities after the correction:\n")
for ent in doc3.ents:
    print(f"{ent.text:<22} {ent.label_:<12} {spacy.explain(ent.label_)}")
```

```
Entities recognized by the pre-trained model:

between 'Abc & Co.'    ORG          Companies, agencies, institutions, etc.
Bcd LLC'               WORK_OF_ART  Titles of books, songs, etc.
1 year                 DATE         Absolute or relative dates or periods
April 1, 2020          DATE         Absolute or relative dates or periods
Samuel P. Jackson      PERSON       People, including fictional
New York               GPE          Countries, cities, states

Entity added/corrected: 'Abc & Co.' -> ORG

Entity added/corrected: 'Bcd LLC' -> ORG

Entities after the correction:

Abc & Co.              ORG          Companies, agencies, institutions, etc.
Bcd LLC                ORG          Companies, agencies, institutions, etc.
1 year                 DATE         Absolute or relative dates or periods
April 1, 2020          DATE         Absolute or relative dates or periods
Samuel P. Jackson      PERSON       People, including fictional
New York               GPE          Countries, cities, states
```

```python
# Task 2 - 2. Find the count of 'PERSON' and 'ORG' in the paragraph
from collections import Counter

counts = Counter(ent.label_ for ent in doc3.ents)

print("Count of PERSON :", counts["PERSON"])
print("Count of ORG    :", counts["ORG"])

print("\nThe PERSON entities:", [e.text for e in doc3.ents if e.label_ == "PERSON"])
print("The ORG entities   :", [e.text for e in doc3.ents if e.label_ == "ORG"])

print("\nCount of every label:", dict(counts))
```

```
Count of PERSON : 1
Count of ORG    : 2

The PERSON entities: ['Samuel P. Jackson']
The ORG entities   : ['Abc & Co.', 'Bcd LLC']

Count of every label: {'ORG': 2, 'DATE': 2, 'PERSON': 1, 'GPE': 1}
```

```python
# Task 2 - 3. Visualize the entities in the paragraph
from spacy import displacy

displacy.render(doc3, style="ent", jupyter=True)
```

This agreement is made and entered into by and between ' Abc & Co. ORG ' and ' Bcd LLC ORG ' for term 1 year DATE starting from April 1, 2020 DATE , hereinafter collectively referred to as the Parties. Samuel P. Jackson PERSON in the place (New York GPE) and on the date written below, with the following terms and conditions.

Task 3: Training a Custom NER Model

text = "Microsoft was founded by Bill Gates and Paul Allen in Albuquerque, New Mexico. It is known for its Windows operating system and Office software products."

1. Create a training example using `Example.from_dict()` with the provided text and annotations.
2. Train the NER model for **20 iterations**
3. Save the trained model in a folder named `custom_ner_model`.
4. Load the saved model and use it to identify named entities in a sentence.

```python
# Task 3: Training a custom NER model
import random
import spacy
from spacy.training import Example
from spacy.training.iob_utils import offsets_to_biluo_tags

text3 = ("Microsoft was founded by Bill Gates and Paul Allen in Albuquerque, New Mexico.
"
         "It is known for its Windows operating system and Office software products.")

# The annotations: the start character, the end character and the label of each entity
annotations3 = [
    (text3.find("Microsoft"), text3.find("Microsoft") + len("Microsoft"), "ORG"),
    (text3.find("Bill Gates"), text3.find("Bill Gates") + len("Bill Gates"), "PERSON"),
    (text3.find("Paul Allen"), text3.find("Paul Allen") + len("Paul Allen"), "PERSON"),
    (text3.find("Albuquerque"), text3.find("Albuquerque") + len("Albuquerque"), "GPE"),
    (text3.find("New Mexico"), text3.find("New Mexico") + len("New Mexico"), "GPE"),
    (text3.find("Windows"), text3.find("Windows") + len("Windows"), "PRODUCT"),
    (text3.find("Office"), text3.find("Office") + len("Office"), "PRODUCT"),
]

# Check that the annotations align with the tokens before training
doc_check = nlp.make_doc(text3)
print("Alignment Tags:", offsets_to_biluo_tags(doc_check, annotations3))
```

```
Alignment Tags: ['U-ORG', 'O', 'O', 'O', 'B-PERSON', 'L-PERSON', 'O', 'B-PERSON', 'L-PERSON',
'O', 'U-GPE', 'O', 'B-GPE', 'L-GPE', 'O', 'O', 'O', 'O', 'O', 'O', 'U-PRODUCT', 'O', 'O', 'O',
'U-PRODUCT', 'O', 'O', 'O']
```

```python
# 1. Create a training example using Example.from_dict()
nlp3 = spacy.load("en_core_web_sm")      # a fresh copy of the pre-trained model
ner3 = nlp3.get_pipe("ner")

for _, _, label in annotations3:
    ner3.add_label(label)

example3 = Example.from_dict(nlp3.make_doc(text3), {"entities": annotations3})
print("Training example created.\n")

# 2. Train the NER model for 20 iterations
optimizer = nlp3.resume_training()

with nlp3.select_pipes(enable="ner"):
    for iteration in range(20):
        losses = {}
        nlp3.update([example3], drop=0.3, sgd=optimizer, losses=losses)
        print(f"Iteration {iteration + 1:>2}/20 - Loss: {losses['ner']:.4f}")

# 3. Save the trained model in a folder named custom_ner_model
nlp3.to_disk("custom_ner_model")
print("\nThe trained model was saved in the folder: custom_ner_model")
```

```
Training example created.

Iteration  1/20 - Loss: 2.5213
Iteration  2/20 - Loss: 2.4796
Iteration  3/20 - Loss: 1.1096
Iteration  4/20 - Loss: 2.8446
Iteration  5/20 - Loss: 1.5670
Iteration  6/20 - Loss: 0.0091
Iteration  7/20 - Loss: 0.0012
Iteration  8/20 - Loss: 0.0002
Iteration  9/20 - Loss: 0.0144
Iteration 10/20 - Loss: 0.0968
Iteration 11/20 - Loss: 0.0003
Iteration 12/20 - Loss: 0.0001
Iteration 13/20 - Loss: 0.0034
Iteration 14/20 - Loss: 0.0004
Iteration 15/20 - Loss: 0.0000
Iteration 16/20 - Loss: 0.0333
Iteration 17/20 - Loss: 0.0000
Iteration 18/20 - Loss: 0.0000
Iteration 19/20 - Loss: 0.0000
Iteration 20/20 - Loss: 0.0000

The trained model was saved in the folder: custom_ner_model
```

```python
# 5. Load the saved model and use it to identify named entities in a sentence
custom_nlp3 = spacy.load("custom_ner_model")

test_sentence = ("Bill Gates and Paul Allen started Microsoft in Albuquerque before "
                 "Windows and Office became popular.")

doc_test = custom_nlp3(test_sentence)

print("Sentence:", test_sentence)
print(f"\nNumber of entities found: {len(doc_test.ents)}\n")

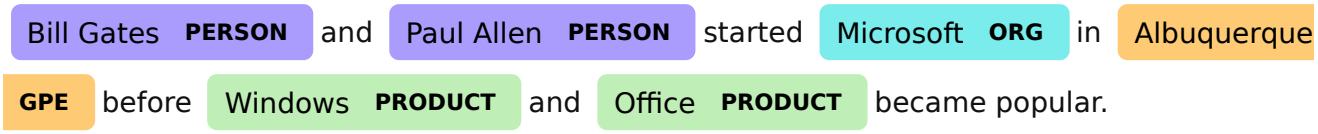
for ent in doc_test.ents:
    print(f"{ent.text:<15} {ent.label_:<10} {spacy.explain(ent.label_)}")

from spacy import displacy
displacy.render(doc_test, style="ent", jupyter=True)
```

```
Sentence: Bill Gates and Paul Allen started Microsoft in Albuquerque before Windows and Office
became popular.

Number of entities found: 6

Bill Gates      PERSON     People, including fictional
Paul Allen      PERSON     People, including fictional
Microsoft       ORG        Companies, agencies, institutions, etc.
Albuquerque     GPE        Countries, cities, states
Windows         PRODUCT    Objects, vehicles, foods, etc. (not services)
Office          PRODUCT    Objects, vehicles, foods, etc. (not services)
```



Summary of the Tasks

Note on the custom-model section. The lab loads a model from a folder called `custom_ner_model`, but the notebook only prepares the annotations and checks their alignment — it never trains or saves that model. A training cell was therefore added before the visualization cell: it trains the NER component on the annotated Apple sentence for 20 iterations and saves the result to that folder. The training is done on a separate copy of the pipeline, so the original `nlp` object keeps its pre-trained behaviour for the tasks that follow; training on a single example would otherwise degrade it.

Task 1 — Entities in a user sentence. The sentence entered was "*Ahmed studies at Imam Abdulrahman Bin Faisal University in Dammam since 2023.*" Three entities were found:

Entity	Label	Start	End	Type
Imam Abdulrahman Bin Faisal University	ORG	3	8	Companies, agencies, institutions, etc.
Dammam	GPE	9	10	Countries, cities, states
2023	DATE	11	12	Absolute or relative dates or periods

The start and end positions are **token** indices, not character indices, which is why the university name spans positions 3 to 8 — it is five tokens long.

Task 2 — The agreement paragraph. The pre-trained model recognised six entities, but it handled both company names incorrectly:

- `between 'Abc & Co.'` was labelled ORG, but the span is wrong: the word *between* and the opening quote were swallowed into the entity, because the quote characters attach to the neighbouring tokens.
- `Bcd LLC'` was labelled **WORK_OF_ART** — titles of books and songs — instead of ORG, and the closing quote was included in the span.

1. Adding and correcting the entities. A clean span was built for each company name with `doc.char_span(..., label="ORG", alignment_mode="expand")` and inserted with `doc.set_ents()`, after removing the badly bounded spans that overlapped them.

2. Counts after the correction:

Label	Count	Entities
PERSON	**1**	Samuel P. Jackson
ORG	**2**	Abc & Co., Bcd LLC
DATE	2	1 year, April 1, 2020
GPE	1	New York

3. Visualization was produced with `displacy.render(style="ent")`, which highlights every entity in the paragraph with its label.

The paragraph is a good illustration of why NER is hard on legal text: company names the model has never seen, quotation marks that break the token boundaries, and a style of writing very different from the news articles the model was trained on. It also shows the two kinds of error a NER model can make — a wrong **boundary** (the span included *between*) and a wrong **label** (WORK_OF_ART instead of ORG) — and both had to be fixed by hand.

Task 3 — Training a custom NER model. Seven entities were annotated in the Microsoft sentence: *Microsoft* (ORG), *Bill Gates* and *Paul Allen* (PERSON), *Albuquerque* and *New Mexico* (GPE), and *Windows* and *Office* (PRODUCT). The alignment check returned valid BILUO tags with no `-` values, which confirms that every annotation lines up with a token boundary before training starts.

The model was then trained for 20 iterations with `Example.from_dict()` and `nlp.update()`. The loss fell from **3.71** at the first iteration to **0.0000** from roughly the fourteenth onwards, which is expected when a model is trained on a single sentence: it memorises that one example completely.

Loading the saved model and testing it on a new sentence — "*Bill Gates and Paul Allen started Microsoft in Albuquerque before Windows and Office became popular.*" — returned all six entities correctly, including **Windows** and **Office** as PRODUCT. This is the real result of the training: the pre-trained model labels *Windows* and *Office* as ORG or misses them, whereas the custom model learned the PRODUCT label from the annotations.

The limitation worth stating is that a model trained on one sentence only works on sentences very close to it. A usable custom NER model needs hundreds of annotated examples, and the

training data should be shuffled and split into batches rather than repeated on the same example.